Completing Structured Arguments in Assumption-based Argumentation (with supplementary material)

Andrei Popescu and Johannes P. Wallner

Graz University of Technology, Graz, Austria
{andrei.popescu,johannes.p.wallner}tugraz.at

Abstract. In their daily use arguments are usually not completely enunciated. That is, we often rely on implicit parts, for example unstated premises, sometimes referred to as enthymemes. Completions of partially stated arguments can favor knowledge engineering processes, where the workload of an engineer can be reduced by suggesting such completions. In this work, we focus on an integral aspect of completing arguments: valid argument structure of a completion. We phrase our results in the formal model of assumption-based argumentation (ABA). Based on an alternative characterization of tree-based arguments in ABA, we provide a declarative approach to compute completions of partial arguments in answer set programming (ASP), including the possibility of preferential reasoning in completions. We empirically evaluate a resulting prototype.

1 Introduction

Reasoning with uncertainty and evolving scenarios is a central aspiration in the design of intelligent systems. Computational argumentation [2, 21] is a paradigm for non-monotonic reasoning that resolves conflicting and possibly incomplete information, with several heterogeneous application avenues [1] such as in legal reasoning [34] and medical reasoning [12, 15].

Key to several such applications are approaches commonly referred to as structured argumentation [14, 32, 4, 22, 26, 29]. In these approaches arguments are instantiated from possibly conflicting knowledge bases and relations between such arguments are identified, such as one argument countering another. Based on the arguments and their relations, argumentation semantics lead to finding rational argumentative conclusions [10, 9].

Considering a recent application for fraud detection at the Dutch National Police [33], constructing and designing adequate knowledge bases is central to applications of computational argumentation. The construction of such knowledge bases, including in particular rules, is usually subsumed under the term "knowledge engineering". Here experienced persons distill their knowledge of domains, processes, and further thoughts into knowledge bases.

Knowledge engineering encompasses various aspects [37], among them in computational argumentation the design of rules that make up arguments. For

instance, in the fraud detection example application, a knowledge engineer might decide that there should be arguments in favor or against fraud, that is concluding fraud or the negation of fraud. Such conclusions do not exist in a vacuum and must be concluded based on rational processes. Arguments can have many components [39], e.g., in major formal approaches to structured argumentation components of arguments are premises or assumptions, rules for derivation, and a conclusion.

In many cases, the design and construction of rules that ultimately lead to useful arguments is likely an incremental process: one can iteratively expand, e.g., on rules that then make up an argument for or against a conclusion. Viewed in a broader sense, arguments might exist only "partially" during the process. This process can be supported by automatic recommendation of possible completions of arguments, to lessen the cognitive load of an engineer, for instance highlighting rules whose bodies cannot be derived using assumptions and suggesting possible additional rules.

In this work, we study the problem of argument completion, and situate our work in the well-known structured argumentation formalism of assumption-based argumentation (ABA) [7, 14]. Connected to the general problem of enthymemes [27, 5, 6], i.e., handling of missing premises of arguments, we in particular focus on the key structure of arguments: they must present full derivations of conclusions (and all parts) based on available assumptions, i.e., arguments have to be a "derivation tree". Such structures can also be found in other structured argumentation formalisms such as ASPIC$^+$ [32].

That is, when completing arguments we in particular require that such completions are actually arguments—satisfying the structural constraint of being derivation trees—and that completions can be automatically computed. This suggests the usage of approaches of declarative problem solving that are both efficient and can handle expressive constraints. Based on recent top-performing systems in computational approaches to ABA and ASPIC$^+$ [31, 30, 28], we focus on answer set programming (ASP) [25, 8, 23] for computing completions of partially specified arguments, which in particular offers capabilities of including declarative specification of preferential information, preferring certain completions over others.

Our main contributions are as follows.

- We formulate argument completion as a computational problem, including constraints such as restricting number and size of learnable rules and incorporation of preferential reasoning.
- In order to be able to work on a representation of arguments amenable to specification in ASP, we provide an alternative representation of tree-based arguments under mild restrictions.
- We show that the general argument completion problem is NP-hard, which further motivates the use of search algorithms such as ASP solvers.
- We provide a prototype in ASP and empirically show promise both in terms of running times and that utilization of preferences can increase the quality of learned arguments.

More details are available in the supplementary material and online[1].

2 Assumption-Based Argumentation

We recall main background of assumption-based argumentation (ABA) [7]. In ABA, the components are a set of atoms $\mathcal{L}$, a set of assumptions $\mathcal{A} \subseteq \mathcal{L}$ chosen from the set of atoms, and rules of the form $h \leftarrow s_1, \ldots, s_n$, with $h, s_1, \ldots, s_n$ atoms. We use shorthands for the head and body of a rule r: $\mathit{head}(r) = h$ and $\mathit{body}(r) = \{s_1, \ldots, s_n\}$. We focus on flat ABA, which can be represented via requiring that h is not an assumption (assumptions cannot be derived). A contrary function defines what is the contrary of an assumption. We call $(\mathcal{L}, \mathcal{R})$ a deductive system.

Definition 1. *An ABA framework is a tuple $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, {}^-)$, where $(\mathcal{L}, \mathcal{R})$ is a deductive system, $\mathcal{A} \subseteq \mathcal{L}$ a non-empty set of assumptions, and ${}^-$ a function mapping assumptions $\mathcal{A}$ to atoms $\mathcal{L}$.*

An atom $x \in \mathcal{L}$ is derivable from a set of assumptions $X \subseteq \mathcal{A}$ using rules $\mathcal{R}$, denoted by $X \vdash_{\mathcal{R}} x$, if $x \in X$ or there is a sequence of rules $(r_1, \ldots, r_n)$ such that $\mathit{head}(r_n) = x$ and for each rule r_i we have $r_i \in \mathcal{R}$ and each atom in the body of r_i is derived from rules earlier in the sequence or is in X, i.e., $\mathit{body}(r_i) \subseteq X \cup \bigcup_{j<i} \{\mathit{head}(r_j)\}$. Note that duplicated rules in this sequence can be directly avoided (the later rule can be removed). The deductive closure for an assumption set X w.r.t. rules $\mathcal{R}$ is defined as $\mathit{Th}_{\mathcal{R}}(X) = \{x \in \mathcal{L} \mid X \vdash_{\mathcal{R}} x\}$.

Let $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, {}^-)$ be an ABA framework. An argument A based on F is a finite labeled rooted tree where

- the root is labeled with some atom $c \in \mathcal{L}$,
- each leaf is labeled by an assumption $a \in \mathcal{A}$ or a dedicated symbol $\top \notin \mathcal{L}$,
- each internal node is labeled with $\mathit{head}(r)$ of a rule $r \in \mathcal{R}$ s.t. the set of labels of children of this node is equal to $\mathit{body}(r)$ or $\top$ if the body is empty (and there are exactly $|\mathit{body}(r)|$ many children).

For such an argument A, we denote the conclusion, i.e., the label of the root, by $\mathit{conc}(A) = c$, the set of all atoms (including assumptions) by $\mathit{atoms}(A)$, the assumptions of A by $\mathit{asm}(A)$, and the set of rules occurring in A by $\mathit{rules}(A)$.

Since we will also make use of a different representation of arguments, we recall a related result next.

Proposition 1 ([19, 20]). *Given an ABA framework $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, {}^-)$ it holds that*

- *if A is an argument in F then $\mathit{asm}(A) \vdash_{\mathit{rules}(A)} \mathit{conc}(A)$ and*
- *if $X \vdash_{\mathcal{R}} c$ there is an argument A' in F with $\mathit{asm}(A') \subseteq X$, $\mathit{rules}(A') \subseteq \mathcal{R}$, and $\mathit{conc}(A') = c$.*

[1] https://gitlab.tugraz.at/krr/aba-completion

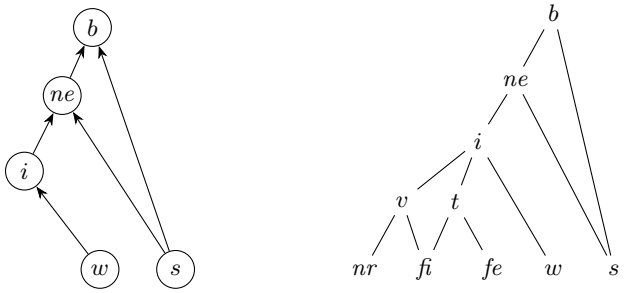


Fig. 1. On the right, an argument is derived in support of protecting against a polarizing social media platform. The dependency graph $G_{\mathcal{S},\mathcal{D}}(V, E)$ of a smaller argument is shown on the left.

Intuitively, if there is an argument, then one can derive the conclusion of the argument, using the rules in the argument and assumptions in the argument. On the other hand, if c is derivable, using rules $\mathcal{R}$ and assumptions X, then there is an argument based on this derivation. However, the argument might not "need" all assumptions or rules in the derivation (in a sense some assumptions or rules might be redundant).

Example 1. Let $\mathcal{L} = \{nr, fi, fe, w, s, v, t, i, ne, b\}$, which represent statements regarding issues of a social media platform, with the following descriptions: b stands for "moderate the platform", ne for "defensive action is needed", i for "legal framework is insufficient", t for "threat is foreign and domestic", v for "platform makes democracy vulnerable", and the last five are assumptions with fi standing for "false information affects voting", nr for "platform does not support democratic oversight", ex for "foreign powers exploit platform", w for "laws are inefficient", and s for "democracies should defend themselves". In Fig. 1 (right) an argument with conclusion b is shown, based on all five assumptions and using rules as indicated in the figure, e.g., $v \leftarrow nr,\ fi$. It holds that $\{nr, fi, fe, w, s\}$ derives b. Moreover, $\{nr, fi, fe\}$ derives v, yet fe is redundant in deriving v.

We provide an expanded example in the supplementary material.

Argumentation semantics [7, 18] can be used to determine the statuses of assumptions or arguments. For instance, arguments can be conflicting, i.e., the conclusion of one argument is the contrary of an assumption in another. A set of arguments defends an argument if all attackers of the argument are attacked by the set of arguments. In this work we mainly focus on argument structures.

3 Completing Partial Arguments

In this section we define a notion of partial arguments and completions of such arguments.

While partial arguments could be defined as subarguments, or subgraphs, of a given argument, for our aim of a computational approach involving declarative

problem solving approaches, we formulate partial arguments and their completions as sets of atoms, rules, and a conclusion. This has the benefit of being more directly amenable for declarative algorithms, and also eases working with such partial arguments, since the exact tree does not need to be specified (but only its contents).

A simplification we apply is that we assume that an argument contains no two rules with the same head. While in principle arguments may have two different rules with the same head, we think this assumption is not a strong restriction for real-world arguments. Moreover, an argument with two different rules for a head (or more) can be transformed into an argument with only one such rule (by choosing one such rule and replacing subarguments which might lead even to fewer required assumptions).

Towards a definition of partial arguments, we first view arguments as triples (S, D, c) with S a set of atoms (including assumptions) $c \in S$ an atom, and D a set of rules. We will assume that the rules in D are over S, i.e., for each $r \in D$ we find that $head(r) \in S$ and $body(r) \subseteq S$.

Given an argument it is straightforward to extract (S, D, c): collect all atoms in S, the conclusion in c, and rules in D. The other direction is less straightforward, since not every such triple corresponds to an argument, in a similar vein as in Proposition 1. For instance, one might be able to derive c via rules D using assumptions in S, however there might be atoms in S or rules in D that are unused in the argument.

Example 2. Consider a triple $(S, D, c) = (\{s, w, ne, i, b, a\}, \{(i \leftarrow w), (b \leftarrow ne, s), (ne \leftarrow i, s)\}, b)$ with $\{s, w, a\}$ being assumptions. This triple does not represent an argument since assumption a is unused in the derivation. Analogously, a triple (S, D, c) would not represent an argument if some rule were unused.

For utilizing triples as a representation for arguments, we define conditions on such triples that ensure that they represent arguments. We will make use of an auxiliary definition that can be seen as a dependency graph in a given ABA framework [11].

Definition 2. *Given a set of atoms $\mathcal{L}$ and a set of rules $\mathcal{R}$ over $\mathcal{L}$, the directed dependency graph $G_{\mathcal{L},\mathcal{R}}(V, E)$ is composed of $V = \mathcal{L}$ and $(x, y) \in E$ if and only if there is a rule $r \in \mathcal{R}$ with $x \in body(r)$ and $head(r) = y$.*

Example 3. Consider the argument represented by the triple $(\{s, w, ne, i, b\}, \{(i \leftarrow w), (b \leftarrow ne, s), (ne \leftarrow i, s)\}, b)$, $G_{\mathcal{S},\mathcal{D}}(V, E)$ has $V = \{s, w, ne, i, b\}$ and $E = \{(w, i), (ne, b), (s, b), (i, ne), (s, ne)\}$, as shown on the left of Fig. 1.

Based on the graph $G_{\mathcal{L},\mathcal{R}}$, we can define conditions such that triples correspond directly to arguments.

Definition 3. *Let $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$ be an ABA framework and (S, D, c) be a triple such that $S \subseteq \mathcal{L}$, $D \subseteq \mathcal{R}$ over S, and $c \in S$. We say that (S, D, c) represents an argument if*

- *for each* $x \in S$ *we find* $(S \cap \mathcal{A}) \vdash_D x$,
- *for each* $x \in S$ *it holds that there is a path from* x *to* c *in* $G_{S,D}$, *and*
- *for each* $r_1, r_2 \in D$ *with* $r_1 \neq r_2$ *we find* $head(r_1) \neq head(r_2)$.

In words, a triple (S, D, c) represents an argument, if each atom in the argument can be derived using the assumptions and rules in the argument, the conclusion is reachable using the rules as connections in the dependency graph, and finally, no two rules share the same head (see discussion above).

We now show that the above conditions indeed imply that there is an argument based on such a triple.

Theorem 1. *Let* $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$ *be an ABA framework, and* (S, D, c) *be a triple such that* $S \subseteq \mathcal{L}$, $D \subseteq \mathcal{R}$ *over* S *and without two rules sharing a head, and* $c \in S$. *It holds that there is an argument* A *in* F *s.t. no two rules share a head in* A *with* $atoms(A) = S$, $rules(A) = D$, *and* $conc(A) = c$ *if and only if* (S, D, c) *represents an argument.*

Proof. Assume that A is an argument in F without two rules having the same head, with $atoms(A) = S$, $rules(A) = D$, and $conc(A) = c$. It is direct to see that $S \subseteq \mathcal{L}$, $D \subseteq \mathcal{R}$ is over S, and $c \in \mathcal{L}$ (by definition of arguments). Moreover, by previous results [19, 20], see Proposition 1, it holds that $(S \cap \mathcal{A}) \vdash_{\mathcal{R}} x$ for each $x \in S$. Since A is an argument, there is a path from each node in the argument (each label) to the conclusion, and likewise in $G_{S,D}$. By assumption, it holds that no two rules share a head (satisfying the third condition in Definition 3).

For the other direction, suppose that (S, D, c) is given as defined above. Consider an arbitrary atom in S. Since each such atom is derivable from the assumptions in S using rules in D, it holds that each atom is either an assumption or the head of a rule in D. By definition, there is at most one such rule (no two rules with the same head). Thus, for each non-assumption there is exactly one rule with this non-assumption as the head.

Consider the following algorithm to construct an argument A: start with a root node with label c. If c is an assumption, then $S = \{c\}$ and $D = \emptyset$, and A is an argument for c. Otherwise, take the rule with c as the head (there is exactly one). Construct nodes for each body element of this rule, and label these nodes with the body elements. Iteratively continue this process for each child node until termination.

We now show that (i) this process terminates with assumptions in S as leaves and (ii) each element in S and D is used in this algorithm, i.e., there is a node in the argument labeled with an $x \in S$ for each element in S, and for each rule $r \in D$ we find an internal node, or root node, corresponding to the rule (a node labeled with the head and children nodes labeled with the body elements).

To see termination, consider an arbitrary path from the root node (labeled with the conclusion c) "downwards", i.e., always from heads to body elements. This path is uniquely determined by D and S (there is exactly one rule per non-assumption). Since for each atom x we find $(S \cap \mathcal{A}) \vdash_D x$, it holds that there is a derivation $(r_1, \ldots, r_n)$ with $head(r_n) = x$ starting from assumptions in $S \cap \mathcal{A}$. If r_i is in this sequence, then all its body elements are derived earlier. In turn, if

there is a node in the argument labeled with the head or r_i, then all its children nodes are labeled with assumptions or heads of rules r_j with $j < i$. This holds for each path starting from the root node. Thus, each such path terminates with assumptions in the leaves (and no rule is used twice in a path).

Finally, consider an arbitrary atom $x \in S$. To see that this atom occurs in A, recall that the conclusion c is reachable from x in $G_{S,D}$. This means that there is a path from x to c in this graph. Among such paths, consider an arbitrary one. Then the reverse direction is again uniquely determined by S and D (exactly one rule per non-assumption). Then x must occur in A (by going this path in reverse). The same holds for rules, just note that both the bodies of each rule are derivable and each such element (and the head) is part of A. Then A must have a node and children nodes corresponding to this rule.

It also follows from the proof of this result that in such an argument there is no rule which contains an atom x both in the head and body. Apart from this rule being redundant, such a rule would violate the conditions of argument representation.

Based on the triple definition, we define partial arguments directly as such triples, without checking conditions of representation of arguments (since partial arguments are not arguments in general).

Definition 4. *Let $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$ be an ABA framework. A partial argument in F is a triple (S, D, c) with $S \subseteq \mathcal{L}$, $D \subseteq \mathcal{R}$ over S s.t. no two rules have the same head, and $c \in S$.*

Example 4. Consider (S, D, b) with $S = \{nr, fi, fe, s, v, t, i, ne, b\}$, with, as before, $\{nr, fi, fe, s\}$ being assumptions. Let D contain the rules $(v \leftarrow nr, fi)$, $(t \leftarrow fi, fe)$, $(ne \leftarrow i)$, $(ne \leftarrow i, s)$, and $(b \leftarrow ne, s)$, see Fig. 2 (left). As also explicated in the figure, this is a partial argument. Compared to the one in Fig. 1 (right), an assumption and a rule are missing.

Given a partial argument (S, D, c), we say that another triple with the same conclusion (S', D', c) "expands" on the partial argument if $S \subseteq S'$ and $D \subseteq D'$, denoted by $(S, D, c) \sqsubseteq (S', D', c)$. For a set of partial arguments $\mathcal{I}$, this notation extends straightforwardly: $\mathcal{I} \sqsubseteq \mathcal{I}'$ if for each $(S, D, c) \in \mathcal{I}$ there is an $(S', D', c) \in \mathcal{I}'$ s.t. $(S, D, c) \sqsubseteq (S', D', c)$, and vice versa: if $(S', D', c) \in \mathcal{I}'$ then there is an $(S, D, c) \in \mathcal{I}$ with $(S, D, c) \sqsubseteq (S', D', c)$.

Definition 5. *Let $F = (\mathcal{L}, \mathcal{R}, \mathcal{A}, ^-)$ be an ABA framework and $\mathcal{I}$ be a set of partial arguments in F. A completion $\mathcal{I}^C$ of $\mathcal{I}$ is a set of triples such that*

- *$\mathcal{I} \sqsubseteq \mathcal{I}^C$ and*
- *each $(S, D, c) \in \mathcal{I}^C$ represents an argument.*

Note that the triples in $\mathcal{I}^C$ are not restricted to atoms, assumptions, or rules in F. A "completed" argument, represented here as a triple, might use components in F or add different atoms, assumptions, or rules. We say that the components in F are available, while components not in F are learned by an $(S', D', c) \in \mathcal{I}^C$.

Example 5. Continuing Example 4, one possible completion is given by recovering rule $i \leftarrow v, t, w$, which is now a learned rule. Another way to complete the partial argument is to add an atom, say l, and add two rules: $l \leftarrow v, t, s$ and $i \leftarrow l$. We discuss below ways of preferring completions over others.

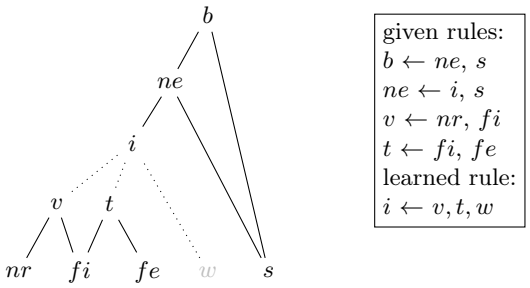


Fig. 2. Partial argument (left), and one possible completion on the right.

Given an arbitrary partial argument, in our general definition, there might not always be a completion that leads to a triple (S, D, c) representing an argument. This can be because every atom in S is already derivable from assumptions in S and rules D. In the latter case either (S, D, c) is already representing an argument or one would need to "remove" parts of S or D to represent an argument, e.g., if some parts do not reach the conclusion (one can view these as arguments with multiple conclusions).

Example 6. Consider the two arguments in Fig. 3. If we combine these two into one triple, with say i as the conclusion, there is no completion, since we cannot add any meaningful rule (all rules going back from i to assumptions are given).

In general, it might be that the existence of completions requires multiple rules to be learned or not having too many restrictions on how to complete a partial argument. Having certain restrictions is useful, and, as we show next, can induce NP-hardness. Consider bounds on the number of learned rules and their bodies. For given two natural numbers $n, k \geq 1$, a learned rule r must have a non-empty body with size at most k: $1 \leq |body(r)| \leq k$. Moreover, the

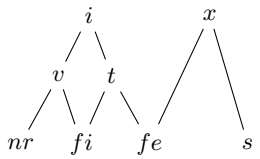


Fig. 3. Two arguments

number of learned rules must be at most n. We restrict to non-empty bodies, since otherwise atoms can essentially be seen as facts.

Theorem 2. *The problem of deciding whether there is a completion for a partial argument over a given ABA framework, and constraints on the number of learnable rules and their size is NP-hard. The problem is in NP if the number of rules that can be learned is given in unary.*

Revisiting Example 5, in addition to restricting learning of components such as rules, choosing which rules to learn can be essential. For this purpose we introduce preferential information of the following form.

Say we learn a rule $h \leftarrow x_1, \ldots, x_n$, in general these atoms do not need to have a connection in some way for such a rule to be learned. With additional information, e.g., that certain atoms in a rule are related, we can make more informed constructions of a rule.

We formalize relatedness of atoms by $T \subseteq 2^{\mathcal{L}}$, i.e., by a set of sets of atoms. We interpret $S \in T$ so that each atom in S is related to the others in S. If a rule r is learned with different "topics", i.e., containing atoms from different relations, we attach a cost for each such diverging topics. Define $T(r) = \{S \in T \mid S \cap body(r) \neq \emptyset\}$. Then the cost of a learned rule is $cost(r) = |\{T(r) \mid T(r) \neq \emptyset\}| - 1$. The task is then to find a completion with minimum cost overall completions. The cost of a completion is the sum of costs of learned rules.

Example 7. Consider again Example 5. If we relate for each "original" rule their bodies as one relation, e.g., $\{v, t, w\}$ in one relation S, then an optimal solution would not learn a rule like $l \leftarrow v, t, s$ since $\{s\}$ and $\{v, t, w\}$ are different relations.

Since we base our computational approaches to complete arguments in declarative problem solving, we can directly state additional optimization statements, such as prioritizing first minimizing the number of rules and then relatedness as defined above. In Example 5, we would prefer to add a single rule (which is necessary) rather than more than one rule.

4 Declarative Approach to Compute Completions

We first recall basics in answer set programming (ASP) [25]. A program π is composed of rules r of the form $b_0 \leftarrow b_1, \ldots, b_k, \mathit{not}\, b_{k+1}, \ldots, \mathit{not}\, b_m$, with each b_i being an atom. We say that a rule is positive if $k = m$ and that a rule is a fact if $m = 0$. A literal is an atom b_i or $\mathit{not}\, b_i$. A rule without head b_0 is called a constraint and a shorthand for $a \leftarrow b_1, \ldots, b_k, \mathit{not}\, b_{k+1}, \ldots, \ \mathit{not}\, b_m, \mathit{not}\, a$ for a fresh a. An atom b_i is $p(t_1, \ldots, t_n)$ with each t_j either a constant or a variable. A program π is ground if π is free of variables. For a non-ground program, let GP be the set of rules obtained by applying all substitutions from the variables to the set of constants appearing in the program. An interpretation I, i.e., a subset of all the ground atoms, satisfies a positive rule $r = h \leftarrow b_1, \ldots, b_k$ iff all positive body elements $b_1, \ldots, b_k$ being in I implies that the head atom is in I. For a program π consisting only of positive rules, let $Cl(\pi)$ be the uniquely

Listing 1.1. Module $\pi_{rule_generation}(n, k)$

```
1  {learnable(1..n)}.
2  {rule(R, Arg)} ← learnable(R), argument(Arg).
3  {rule(R, Arg2)} ← rule(R, Arg), argument(Arg2).
4  {head(R,X)} ← atom(X), learnable(R).
5  {body(R,X)} ← atom(X), learnable(R).
6  {body(R,X)} ← assumption(X), learnable(R).
7  ← #count {X: body(R,X)} > k, learnable(R).
8  ← not body(R,_), learnable(R).
9  ← not head(R,_), learnable(R).
10 ← body(R,X), head(R,X), learnable(R).
11 ← head(R,X), head(R,Y), X != Y, learnable(R).
12 ← rule(R1, Arg), rule(R2, Arg), head(R1, X), head(R2, X), R1 != R2.
```

determined interpretation I that satisfies all rules in π and no subset of I satisfies all rules in π. Interpretation I is an answer set of a ground program π if $I = Cl(\pi^I)$ where $\pi^I = \{(h \leftarrow b_1, \ldots, b_k) \mid (h \leftarrow b_1, \ldots, b_k, not\, b_{k+1}, \ldots, not\, b_m) \in \pi, \{b_{k+1}, \ldots, b_m\} \cap I = \emptyset\}$ is the reduct, and of a non-ground program π if I is an answer set of GP of π.

We make use of ASP predicates to represent a given ABA F and learnable rules or atoms. All given atoms x are defined as $\mathbf{atom}(x)$.

A given partial argument a_i with triple (S, D, c) is represented in ASP as $\mathbf{argument}(a_i)$, $\mathbf{rule}(r, a_i)$ if $r \in D$, $\mathbf{inarg}(x, a_i)$ for $x \in S$, $\mathbf{conclusion}(c, a_i)$, $\mathbf{head}(r, h)$ if $h = head(r)$, and $\mathbf{body}(r, b)$ if $b \in body(r)$.

We first describe our ASP encoding for rule generation in Listing 1.1 with parameters n and k. The encoding can generate up to n new rules to complete arguments (Line 1). Each learnable rule is assigned to some or to no argument, and has a body, with one or more body atoms assigned to the rule non-deterministically. Rules from an argument might be used in another (Line 3) Constraints from Lines 7 to 12 enforce an upper bound of the body elements for learned rules, such that each learnable rule has at least one body atom and one different atom for the head, as well as satisfying the condition of one argu-

Listing 1.2. Module $\pi_{atom_generation}(l, u)$

```
1 pool_atom(l..u).
2 {atom(A)} ← pool_atom(A).
3 {assumption(A)} ← pool_asm(A).
4 inarg(X,Arg) ← rule(R,Arg), head(R,X), learnable(R).
5 inarg(X,Arg) ← rule(R,Arg), body(R,X), learnable(R).
6 inarg(X,Arg) ← rule(R,Arg), head(R,X).
7 inarg(X,Arg) ← rule(R,Arg), body(R,X).
```

Listing 1.3. Module π_{deriv}

```
1  derivable(X,Arg) ← assumption(X), inarg(X, Arg).
2  derivable(X,Arg) ← head(R,X),triggered(R, Arg), rule(R, Arg), inarg(X,Arg).
3  triggered(R,Arg) ← rule(R,Arg), derivable(X,Arg) : body(R,X).
4  ← atom(X), inarg(X, Arg), not derivable(X, Arg).
```

Listing 1.4. Module π_{reach}

```
1  start(X, Arg) ← assumption(X), inarg(X, Arg).
2  start(X, Arg) ← head(R,X), not conclusion(X,Arg), inarg(X, Arg), rule(R, Arg).
3  reach(X,Y, Arg) ← head(R,Y), body(R,X), rule(R, Arg).
4  reach(X,C, Arg) ← reach(X,D, Arg), body(R,D), head(R,C), rule(R, Arg).
5  ← conclusion(I,Arg),start(X,Arg), not reach(X,I,Arg),inarg(X,Arg),inarg(I,Arg).
```

ment not containing two rules with the same head. To complete learnable rules, we use atoms for body elements and non-assumptions for the head.

For potentially using atoms not in the given partial arguments, we use Listing 1.2. Our encoding has a reserve of extra atoms in the pool (Line 1), and non-deterministically chooses atoms to be used in Lines 2 and 3. These atoms may have been used to "fill" learnable rules, which have already been assigned to some argument(s). Module $\pi_{atom_generation}$ extends the information available about rules, to atoms, by generating the **inarg** predicate in Lines 4 to 7.

Module π_{deriv} in Listing 1.11 ensures that all atoms in an argument are derivable via the rules in argument (Line 4), inspired by [31]. The derivation starts with the assumptions in the argument (Line 1), and derives atoms which are the head of some rule whose body is fully derived already in the argument. This is achieved in Line 3 by marking as "triggered" any rule whose body is fully derived, and in Line 2 by deriving the atoms in the head of such rules.

In Module π_{reach} (Listing 1.4), we ensure that arguments are connected by checking, within each argument, that its conclusion is reachable via the rules in the argument, from every assumption in the argument, and from every atom head of a rule in the argument.

Listing 1.5 encodes preferences. We use a weak constraint for implementing preferences over rules chosen in a completion (Line 3), and the **related**$(rel, atom)$ predicate for representing relations. For each learned rule in the completion, the

Listing 1.5. Module π_{pref}

```
1  in_rule(R,X) ← atom(X), 1 body(R,X); head(R,X)}, learnable(R).
2  in_rule(R,X) ← assumption(X), 1 {body(R,X); head(R,X)}, learnable(R).
3  ⇐ related(T1, X), related(T2, Y), T1!=T2, learnable(R), in_rule(R,X),
       in_rule(R,Y). [1@1, R,T1]
```

ASP solver incurs a cost of one for the rule and each relation in the rule, if this rule contains atoms associated with more than one relation.

5 Experimental Evaluation

In this section we report on an empirical evaluation of our ASP-based prototype.

Instance Generation To the best of our knowledge, there is currently no real-world benchmark set for ABA directly amenable to partial arguments. We used ABA frameworks from the International Competition on Computational Models of Argumentation (ICCMA) 2023 [28]. We included instances with 100, 500, and 2000 atoms, containing 10 to 600 assumptions. For each ABA framework, we generate a problem instance suitable for our prototype by first constructing arguments within a 300 seconds time limit. For subsets of the assumptions of size 1 to 5, we constructed arguments using all of the assumptions in the subset, requiring that they have a unique last derivation (a unique conclusion), and discarding trivial arguments with only one rule. Each instance suitable for our system is generated by randomly picking up to 10 constructed arguments for the respective ABA framework, and uniformly removing a rule with $\frac{1}{2}$ probability. For relatedness, we considered two variants: *i.* by fixing the number of relatedness relations to 10 and randomly assigning body elements to these, and *ii.* by iterating over all derivation rules in the selected arguments, and assigning each body element that has not been previously assigned, to a fresh relation. Overall, this resulted in 217 instances. The max. number of learnable rules and max. body size are parameters, and in our experiments are set to 20 and 5 respectively.

Evaluation All experiments were carried out on a Linux 8GB i5 Intel machine, with a 5 minutes time limit and with clingo 5.6.2 [24]. We compare the performance and quality on the two datasets, one with randomly assigned relatedness relations, and the other with rule-based relatedness. For the quality of each completion of a set of partial arguments, we compare the learned rules to their equivalents (same head) in the originally constructed arguments, and we call the distance of the completion from the originally constructed arguments the sum of the count of atoms appearing in one rule but not the other.

In Table 1 we show the count of solved instances, unsolved instances in terms of timeouts, the average distance from the originally constructed arguments and their median running time. Across all instances we see an improvement in terms of number of solved instances, and a decrease in timeouts when using relatedness assigned based on the derivation rules of the selected arguments from which an instance was generated. We show the average distance separately in Table 2, computed only on the instances solved with the two types of relatedness. On these we see decreased distance overall of completions from the originally selected arguments. Regarding runtime, we see complementary results for the instances based on the 100-atoms ABA frameworks (where most instances are solved with both relatedness assignments), i.e. one relatedness assignment improves runtime over the other. More details can be found in the supplementary material.

Table 1. Comparison of random relatedness and rule-based relatedness.

Pref	atoms	#total	**Solved**	Avg. dist.	TOs	median (s)
Rand. relatedness	100	64	52	8.23	12	2.191
	500	77	35	13.65	42	14.28
	2000	76	19	14.15	57	9.78
Rule-based relatedness	100	64	**55**	7.09	**9**	1.96
	500	77	**46**	15.13	**31**	26.18
	2000	76	**42**	18.88	**34**	23.33

6 Related Works

Learning concepts generalizing over specific data is a major aim of AI, and it is pursued, e.g., in the form of learning a general function in statistical approaches or a logic program, such as in inductive logic programming (ILP) [13]. In ABA learning [17, 35, 38, 16] the authors use concepts borrowed from ILP to learn an ABA framework that semantically covers a set of given positive examples, and does not cover any of a given set of negative examples. A notion of repair of ABA frameworks was recently proposed [36] which aims to modify an ABA framework to achieve consistency. Argument mining and enthymemes were also recently studied [3] in the context of logical argumentation. In contrast, we focus on completing arguments in ABA, and provide both a declarative approach and foundational properties.

7 Conclusions

We studied the computational problem of completing partial arguments. We provided a characterization of representation of tree-based arguments on sets, and we showed that in general this problem is NP-hard. We presented a prototype of implementing completion in ASP, and an empirical evaluation of this prototype.

For future work, we think that our approach can be fruitfully combined with machine learning approaches so that the result is, e.g., neuro-symbolic. For instance, one can utilize learning approaches to learn preferential information or available rules, and apply our ASP-based implementation, or another direction is to learn how to complete arguments, where our approach could be compiled to act as constraints in learning so that only actual arguments are learned.

Table 2. Average distance of the completion.

#atoms #solved	100 50	500 32	2000 18
Avg. dist. (Rand. r.)	7.92	**12.65**	13.50
Avg. dist. (Rule-based r.)	**6**	13.5	**12.72**

Acknowledgments. This research was funded in whole or in part by the Austrian Science Fund (FWF) P35632. For open access purposes, the authors have applied a CC BY public copyright license to any author accepted manuscript version arising from this submission.

References

1. Atkinson, K., Baroni, P., Giacomin, M., Hunter, A., Prakken, H., Reed, C., Simari, G.R., Thimm, M., Villata, S.: Towards artificial argumentation. AI Mag. **38**(3), 25–36 (2017)
2. Baroni, P., Gabbay, D., Giacomin, M., van der Torre, L. (eds.): Handbook of Formal Argumentation. College Publications (2018)
3. Ben-Naim, J., David, V., Hunter, A.: Understanding enthymemes in argument maps: Bridging argument mining and logic-based argumentation. CoRR **abs/2408.08648** (2024)
4. Besnard, P., Hunter, A.: A review of argumentation based on deductive arguments. In: Baroni, P., Gabbay, D., Giacomin, M., van der Torre, L. (eds.) Handbook of Formal Argumentation, chap. 9, pp. 437–484. College Publications (2018)
5. Black, E., Hunter, A.: Using enthymemes in an inquiry dialogue system. In: Padgham, L., Parkes, D.C., Müller, J.P., Parsons, S. (eds.) Proc. AAMAS. pp. 437–444. IFAAMAS (2008)
6. Black, E., Hunter, A.: A relevance-theoretic framework for constructing and deconstructing enthymemes. J. Log. Comput. **22**(1), 55–78 (2012)
7. Bondarenko, A., Dung, P.M., Kowalski, R.A., Toni, F.: An abstract, argumentation-theoretic approach to default reasoning. Artif. Intell. **93**, 63–101 (1997)
8. Brewka, G., Eiter, T., Truszczynski, M.: Answer set programming at a glance. Communications of the ACM **54**(12), 92–103 (2011)
9. Caminada, M.: Rationality postulates: Applying argumentation theory for nonmonotonic reasoning. In: Baroni, P., Gabbay, D., Giacomin, M., van der Torre, L. (eds.) Handbook of Formal Argumentation, chap. 15, pp. 771–795. College Publications (2018)
10. Caminada, M., Amgoud, L.: On the evaluation of argumentation formalisms. Artificial Intelligence **171**(5-6), 286–310 (2007)
11. Craven, R., Toni, F.: Argument graphs and assumption-based argumentation. Artif. Intell. **233**, 1–59 (2016)
12. Craven, R., Toni, F., Cadar, C., Hadad, A., Williams, M.: Efficient argumentation for medical decision-making. In: Brewka, G., Eiter, T., McIlraith, S.A. (eds.) Proc. KR. pp. 598–602. AAAI Press (2012)
13. Cropper, A., Dumancic, S.: Inductive logic programming at 30: A new introduction. J. Artif. Intell. Res. **74**, 765–850 (2022)
14. Čyras, K., Fan, X., Schulz, C., Toni, F.: Assumption-based argumentation: Disputes, explanations, preferences. In: Baroni, P., Gabbay, D., Giacomin, M., van der Torre, L. (eds.) Handbook of Formal Argumentation, chap. 7, pp. 365–408. College Publications (2018)
15. Cyras, K., Oliveira, T., Karamlou, A., Toni, F.: Assumption-based argumentation with preferences and goals for patient-centric reasoning with interacting clinical guidelines. Argument Comput. **12**(2), 149–189 (2021)

16. De Angelis, E., Proietti, M., Toni, F.: ABA learning via ASP. In: Pontelli, E., Costantini, S., Dodaro, C., Gaggl, S.A., Calegari, R., d'Avila Garcez, A.S., Fabiano, F., Mileo, A., Russo, A., Toni, F. (eds.) Proceedings 39th International Conference on Logic Programming, ICLP 2023, Imperial College London, UK, 9th July 2023 - 15th July 2023. EPTCS, vol. 385, pp. 1–8 (2023)
17. De Angelis, E., Proietti, M., Toni, F.: Learning brave assumption-based argumentation frameworks via ASP. In: Endriss, U., Melo, F.S., Bach, K., Diz, A.J.B., Alonso-Moral, J.M., Barro, S., Heintz, F. (eds.) Proc. ECAI. Frontiers in Artificial Intelligence and Applications, vol. 392, pp. 3445–3452. IOS Press (2024)
18. Dung, P.M.: On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. Artif. Intell. **77**(2), 321–358 (1995)
19. Dung, P.M., Kowalski, R.A., Toni, F.: Dialectic proof procedures for assumption-based, admissible argumentation. Artif. Intell. **170**(2), 114–159 (2006)
20. Dung, P.M., Toni, F., Mancarella, P.: Some design guidelines for practical argumentation systems. In: Baroni, P., Cerutti, F., Giacomin, M., Simari, G.R. (eds.) Proc. COMMA. Frontiers in Artificial Intelligence and Applications, vol. 216, pp. 183–194. IOS Press (2010)
21. Gabbay, D., Giacomin, M., Simari, G.R., Thimm, M. (eds.): Handbook of Formal Argumentation, vol. 2. College Publications (2021)
22. García, A.J., Simari, G.R.: Argumentation based on logic programming. In: Baroni, P., Gabbay, D., Giacomin, M., van der Torre, L. (eds.) Handbook of Formal Argumentation, chap. 8, pp. 409–435. College Publications (2018)
23. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Answer Set Solving in Practice. Synthesis Lectures on Artificial Intelligence and Machine Learning, Morgan & Claypool Publishers (2012)
24. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Multi-shot ASP solving with clingo. Theory Pract. Log. Program. **19**(1), 27–82 (2019)
25. Gelfond, M., Lifschitz, V.: The stable model semantics for logic programming. In: Proc. ICLP/SLP. pp. 1070–1080. MIT Press (1988)
26. Gordon, T.F., Prakken, H., Walton, D.: The Carneades model of argument and burden of proof. Artif. Intell. **171**(10-15), 875–896 (2007)
27. Hunter, A.: Real arguments are approximate arguments. In: Proc. AAAI, July 22-26, 2007, Vancouver, British Columbia, Canada. pp. 66–71. AAAI Press (2007)
28. Järvisalo, M., Lehtonen, T., Niskanen, A.: ICCMA 2023: 5th international competition on computational models of argumentation. Artif. Intell. **342**, 104311 (2025)
29. Kakas, A.C., Moraitis, P., Spanoudakis, N.I.: *GORGIAS*: Applying argumentation. Argument Comput. **10**(1), 55–81 (2019)
30. Lehtonen, T., Wallner, J.P., Järvisalo, M.: An answer set programming approach to argumentative reasoning in the ASPIC+ framework. In: Calvanese, D., Erdem, E., Thielscher, M. (eds.) Proc. KR. pp. 636–646 (2020)
31. Lehtonen, T., Wallner, J.P., Järvisalo, M.: Declarative algorithms and complexity results for assumption-based argumentation. J. Artif. Intell. Res. **71**, 265–318 (2021)
32. Modgil, S., Prakken, H.: Abstract rule-based argumentation. In: Baroni, P., Gabbay, D., Giacomin, M., van der Torre, L. (eds.) Handbook of Formal Argumentation, chap. 6, pp. 287–364. College Publications (2018)
33. Odekerken, D., Borg, A., Bex, F.: Estimating stability for efficient argument-based inquiry. In: Prakken, H., Bistarelli, S., Santini, F., Taticchi, C. (eds.) Proc. COMMA. Frontiers in Artificial Intelligence and Applications, vol. 326, pp. 307–318. IOS Press (2020)

34. Prakken, H., Sartor, G.: Law and logic: A review from an argumentation perspective. Artif. Intell. **227**, 214–245 (2015)
35. Proietti, M., Toni, F.: Learning assumption-based argumentation frameworks. In: Muggleton, S.H., Tamaddoni-Nezhad, A. (eds.) Proc. ILP. Lecture Notes in Computer Science, vol. 13779, pp. 100–116. Springer (2022)
36. Rapberger, A., Ulbricht, M.: Repairing assumption-based argumentation frameworks. In: Marquis, P., Ortiz, M., Pagnucco, M. (eds.) Proc. KR (2024)
37. Schreiber, G.: Knowledge engineering. In: van Harmelen, F., Lifschitz, V., Porter, B.W. (eds.) Handbook of Knowledge Representation, Foundations of Artificial Intelligence, vol. 3, pp. 929–946. Elsevier (2008)
38. Tirsi, C., Proietti, M., Toni, F.: Abalearn: An automated logic-based learning system for ABA frameworks. In: Basili, R., Lembo, D., Limongelli, C., Orlandini, A. (eds.) Proc. AIxIA. Lecture Notes in Computer Science, vol. 14318, pp. 3–16. Springer (2023)
39. Toulmin, S.E.: The Uses of Argument, Updated Edition. Cambridge University Press (2008)

In this appendix we provide proof details not in the main paper, more examples, and more details on the experimental evaluation.

A Proofs

Proof (Proof of Theorem 2). For membership, if the number of rules is given in unary, one can in polynomial time guess for each learnable rule a head and a set of atoms (subset of the given ones), and check whether the resulting triple (S, D, c) represents an argument (available rules can be guessed to be included or not).

For hardness, consider a reduction from the problem of deciding whether a given Boolean formula $\phi = c_1 \wedge \cdots \wedge c_n$ in conjunctive normal form is satisfiable. For the sake of an easier presentation, we assume that each clause has at most three literals (this is not critical). Let P be the set of Boolean variables in ϕ and each clause represented as a set of literals.

Construct the partial argument with a fresh atom f as the conclusion. First, construct $n \cdot 3$ rules, and each such rule $r^j_{c_i}$ with head c_i and the body a single literal from the clause c_i. Then construct the following rules: for each $x \in P$ let r_x be the rule that has c_x in the head and the body x and r_x the same rule but with $\neg x$ as the body.Then construct a single rule with f in the head and the body is the set $\{c_1, \ldots, c_n\} \cup \{c_x \mid x \in P\}$. Add a single assumption a. Set the set of atoms to be $\{a, f, c_1, \ldots, c_n\} \cup \{c_x, x, \neg x \mid x \in P\}$. Note that "$\neg x$" is a normal atom. Finally, fix that the rule with f in the head must be part of the partial argument, while the others are available.

By requiring that one can learn $|P|$ many rules with a body of size one, we can show that ϕ is satisfiable if and only if there is a completion of the given partial argument.

Assume that ϕ is satisfiable. Then there is a satisfying truth value assignment assigning each $x \in P$ either true or false. For each such variable assignment construct a rule $x \leftarrow a$ if x is assigned to true or $\neg x \leftarrow a$ if x is assigned to false. For each clause the satisfying truth value assignment satisfies at least one literal. If there are multiple, choose one. Then choose the corresponding available rule. The resulting set forms an argument: from assumption a we can derive the "chosen" literal (e.g., x or $\neg x$) and then for each clause a rule that derives the clause. The "additional" atoms c_x can be directly derived by choosing the corresponding rule that uses the chosen literal.

For the other direction, assume that there is a completion. Then the learnt rules must for each $x \in P$ either have x, $\neg x$ or c_x in the head. Otherwise, one of the c_x would not be derived and the rule with f in the head would not be usable in an argument. Thus, no rule with a c_i in the head is learnt. Then the only way for these to be derived is via one of the available rules, and each of them corresponds to a way to satisfy the clause. Finally, note that for each $x \in P$ it cannot be the case that both x and $\neg x$ occur in heads of learnt rules (otherwise, some other $c_{x'}$ would not be derived). From the head of learnt rules a satisfying truth value assignment of ϕ can be constructed.

B Expanded Example

Here we provide an expanded example and its ASP encoding and completion in ASP.

B.1 Arguments' Description

For the running example we can construct the two largest arguments starting from the two 5-elements assumption sets $\{nr, fi, fe, w, s\}$, and $\{fr, au, ca, re, o\}$ respectively.

Table 3. ABA Framework: Ban Position

Component	**Description**
Atoms	(b) `ban`: ban the platform
	(t) `thrt`: threat is foreign and domestic
	(v) `vuln`: platform makes democracy vulnerable
	(i) `insf`: legal framework is insufficient
	(ne) `need`: defensive action is required
Assumptions	(nr) `nreg`: platform refuses democratic oversight
	(fi) `finf`: false information affects voting
	(fe) `fexp`: foreign powers exploit the platform
	(w) `weak`: laws are ineffective
	(s) `self`: democracies should defend themselves
Rules	`ban` $\leftarrow$ `need, self`
	`need` $\leftarrow$ `insf, self`
	`vuln` $\leftarrow$ `finf, nreg`
	`thrt` $\leftarrow$ `finf, fexp`
	`insf` $\leftarrow$ `vuln, thrt, weak`
Contrary	contrary(`self`) = `nban`

Table 4. ABA Framework: No-Ban Position

Component	Description
Atoms	(nb) `nban`: do not ban the platform (ce) `cens`: banning implies censorship (d) `deal`: citizens can deal with disinformation (al) `alts`: alternative solutions exist (ex) `exss`: banning is a disproportionate response
Assumptions	(fr) `free`: access to platforms is part of free speech (au) `auth`: banning is often actuated by authoritarian regimes (ca) `capb`: democratic citizens can handle false content (re) `regl`: regulation is a better alternative (o) `over`: government overreach inhibits free speech

Rules	`nban` ← `exss`
	`exss` ← `cens, alts`
	`alts` ← `deal, regl`
	`cens` ← `auth, over`
	`deal` ← `free, capb`

Contrary	contrary(`free`) = `ban`

B.2 ABA Framework and Generated Problem Instance

Listing 1.6. Running Example - ABA Framework

```
1  assumption(finf). assumption(nreg). assumption(fexp).
2  assumption(weak). assumption(self).
3  atom(ban). atom(vuln). atom(thrt). atom(insf). atom(need).
4  rule(r1,argban). head(r1,vuln). body(r1,finf). body(r1,nreg).
5  rule(r2,argban). head(r2,thrt). body(r2,finf). body(r2,fexp).
6  rule(r3,argban). head(r3,insf). body(r3,vuln). body(r3,thrt). body(r3,weak).
7  rule(r5,argban). head(r5,need). body(r5,insf).
8  rule(r6,argban). head(r6,ban). body(r6,need). body(r6,self).
9
10 assumption(free). assumption(auth). assumption(capb).
11 assumption(regl). assumption(over).
12 atom(nban). atom(deal). atom(cens). atom(alts). atom(exss).
13 rule(r7,argNban). head(r7,deal). body(r7,free). body(r7,capb).
14 rule(r8,argNban). head(r8,cens). body(r8,auth). body(r8,over).
15 rule(r9,argNban). head(r9,alts). body(r9,deal). body(r9,regl).
16 rule(r10,argNban). head(r10,exss). body(r10,cens). body(r10,alts).
17 rule(r11,argNban). head(r11,nban). body(r11,exss).
```

Listing 1.7. Running Example - Problem Instance (rule-based topics)

```
1  assumption(finf). assumption(nreg). assumption(fexp).
2  assumption(self).
3  atom(ban). atom(vuln). atom(thrt). atom(insf). atom(need).
```

```
4  rule(r1,argban). head(r1,vuln). body(r1,finf). body(r1,nreg).
5  rule(r2,argban). head(r2,thrt). body(r2,finf). body(r2,fexp).
6  rule(r5,argban). head(r5,need). body(r5,insf).
7  rule(r6,argban). head(r6,ban). body(r6,need). body(r6,self).
8
9  assumption(auth). assumption(regl). assumption(over).
10 atom(nban). atom(deal). atom(cens). atom(alts).
11 rule(r8,argNban). head(r8,cens). body(r8,auth). body(r8,over).
12 rule(r9,argNban). head(r9,alts). body(r9,deal). body(r9,regl).
13
14 pool_asm(weak). pool_asm(free). pool_asm(capb).
15 pool_atm(insf). pool_atm(vuln). pool_atm(thrt). pool_atm(deal).
16 pool_atm(exss). pool_atm(cens). pool_atm(alts). pool_atm(nban).
17
18 argument(argban).
19 conclusion(ban,argban).
20 inarg(ban,argban). inarg(vuln,argban). inarg(finf,argban).
21 inarg(nreg,argban). inarg(thrt,argban). inarg(fexp,argban).
22 inarg(need,argban). inarg(insf,argban). inarg(self,argban).
23 argument(argnban).
24 conclusion(nban,argnban).
25 inarg(nban,argnban). inarg(alts,argnban). inarg(deal,argnban).
26 inarg(regl,argnban). inarg(cens,argnban). inarg(auth,argnban).
27 inarg(over,argnban).
```

Listing 1.8. Running Example - rule-based relatedness

```
1  related(t1,finf). related(t1,nreg).
2  related(t2,fexp).
3  related(t3,vuln). related(t3,thrt). related(t3,weak).
4  related(t4,insf). related(t4,self).
5  related(t5,need).
6  related(t6,free). related(t6,capb).
7  related(t7,deal). related(t7,regl).
8  related(t8,auth). related(t8,over).
9  related(t9,cens). related(t9,alts).
10 related(t10,exss).
```

Listing 1.9. Running Example - random relatedness

```
1  %Random preferences
2  related(t4,finf). related(t4,nreg). related(t5,fexp). related(t6,weak).
3  related(t6,self). related(t10,vuln). related(t7,thrt). related(t7,insf).
4  related(t4,need). related(t7,ban). related(t8,free). related(t10,auth).
5  related(t1,capb). related(t6,regl). related(t10,over). related(t5,deal).
6  related(t6,alts). related(t7,cens). related(t6,exss). related(t6,nban).
```

Listing 1.10. Running Example - Completion (rule-based relatedness)

```
1 argument(argban0) argument(argnban0) assumption(finf) assumption(
    nreg) assumption(fexp) assumption(self) assumption(regl)
    assumption(auth) assumption(over) body(r1,finf) body(r1,nreg)
    body(r2,finf) body(r2,fexp) body(r5,insf) body(r5,self) body(r6,need)
    body(r6,self) body(r9,deal) body(r9,regl) body(r8,auth) body(r8,over)
    conclusion(ban,argban0) conclusion(nban,argnban0) derivable(finf,
    argban0) derivable(nreg,argban0) derivable(fexp,argban0) derivable(
    self,argban0) derivable(regl,argnban0) derivable(auth,argnban0)
    derivable(over,argnban0) derivable(vuln,argban0) derivable(thrt,
    argban0) derivable(cens,argnban0) head(r1,vuln) head(r2,thrt) head(r5
    ,need) head(r6,ban) head(r9,alts) head(r8,cens) inarg(ban,argban0)
    inarg(vuln,argban0) inarg(finf,argban0) inarg(nreg,argban0) inarg(thrt,
    argban0) inarg(fexp,argban0) inarg(need,argban0) inarg(insf,argban0)
    inarg(self,argban0) inarg(nban,argnban0) inarg(alts,argnban0) inarg(
    deal,argnban0) inarg(regl,argnban0) inarg(cens,argnban0) inarg(auth,
    argnban0) inarg(over,argnban0) rule(r1,argban0) rule(r2,argban0) rule(
    r5,argban0) rule(r6,argban0) rule(r9,argnban0) rule(r8,argnban0)
    learnable(1) learnable(2) learnable(3) learnable(4) learnable(5)
    learnable(6) learnable(7) learnable(8) learnable(9) learnable(10)
    learnable(11) learnable(12) learnable(13) learnable(14) learnable
    (15) learnable(16) learnable(17) learnable(18) learnable(19)
    learnable(20) head(3,ban) head(1,thrt) head(12,thrt) head(18,thrt)
    head(2,insf) head(10,nban) head(19,deal) head(4,exss) head(14,300)
    head(9,301) head(15,301) head(17,301) head(20,301) head(5,302) head
    (13,302) head(16,302) head(6,303) head(8,303) head(11,303) head
    (7,305) body(4,ban) body(19,ban) body(15,vuln) body(16,vuln) body
    (7,thrt) body(15,thrt) body(9,insf) body(5,nban) body(14,alts) body
    (3,cens) body(10,exss) body(20,300) body(2,301) body(6,301) body
    (11,301) body(14,302) body(2,303) body(4,303) body(11,304) body
    (18,304) body(5,305) assumption(weak) assumption(free)
    assumption(capb) body(13,nreg) body(6,self) body(8,self) body(17,
    self) body(19,regl) body(1,weak) body(12,weak) body(15,weak) body
    (20,weak) rule(2,argban0) rule(8,argban0) rule(15,argban0) rule(3,
    argnban0) rule(4,argnban0) rule(6,argnban0) rule(10,argnban0) rule(13,
    argnban0) rule(14,argnban0) rule(19,argnban0) rule(20,argnban0) inarg
    (301,argban0) inarg(303,argban0) inarg(ban,argnban0) inarg(exss,
    argnban0) inarg(300,argnban0) inarg(301,argnban0) inarg(302,argnban0
    ) inarg(303,argnban0) inarg(weak,argban0) inarg(nreg,argnban0) inarg(
    self,argnban0) inarg(weak,argnban0) derivable(nreg,argnban0)
    derivable(self,argnban0) derivable(weak,argban0) derivable(weak,
    argnban0) derivable(ban,argban0) derivable(need,argban0) derivable(
    insf,argban0) derivable(301,argban0) derivable(303,argban0) derivable
    (nban,argnban0) derivable(alts,argnban0) derivable(deal,argnban0)
    derivable(ban,argnban0) derivable(exss,argnban0) derivable(300,
```

argnban0) **derivable**(301,argnban0) **derivable**(302,argnban0) **derivable**(303,argnban0)

Listing 1.11. Running Example - Completion (random relatedness)

```
1 argument(argban0) argument(argnban0) assumption(finf) assumption(
    nreg) assumption(fexp) assumption(self) assumption(regl)
    assumption(auth) assumption(over) body(r1,finf) body(r1,nreg)
    body(r2,finf) body(r2,fexp) body(r6,need) body(r6,self) body(r5,insf)
    body(r5,self) body(r9,deal) body(r9,regl) body(r8,auth) body(r8,over)
    conclusion(ban,argban0) conclusion(nban,argnban0) derivable(finf,
    argban0) derivable(nreg,argban0) derivable(fexp,argban0) derivable(
    self,argban0) derivable(regl,argnban0) derivable(auth,argnban0)
    derivable(over,argnban0) derivable(vuln,argban0) derivable(thrt,
    argban0) derivable(cens,argnban0) head(r1,vuln) head(r2,thrt) head(r6
    ,ban) head(r5,need) head(r9,alts) head(r8,cens) inarg(ban,argban0)
    inarg(vuln,argban0) inarg(finf,argban0) inarg(nreg,argban0) inarg(thrt,
    argban0) inarg(fexp,argban0) inarg(need,argban0) inarg(insf,argban0)
    inarg(self,argban0) inarg(nban,argnban0) inarg(alts,argnban0) inarg(
    deal,argnban0) inarg(regl,argnban0) inarg(cens,argnban0) inarg(auth,
    argnban0) inarg(over,argnban0) rule(r1,argban0) rule(r2,argban0) rule(
    r6,argban0) rule(r5,argban0) rule(r9,argnban0) rule(r8,argnban0)
    learnable(1) learnable(2) learnable(8) learnable(9) learnable(11)
    learnable(16) learnable(17) learnable(20) head(11,insf) head(17,insf)
    head(16,nban) head(20,nban) head(8,deal) head(1,302) head(2,304)
    head(9,304) body(2,vuln) body(17,thrt) body(9,insf) body(20,alts)
    body(11,cens) body(17,cens) body(8,302) body(8,304) body(16,304)
    body(17,304) assumption(weak) assumption(free) assumption(capb)
    body(8,fexp) body(20,self) body(20,regl) body(1,weak) body(20,weak)
     rule(2,argban0) rule(17,argban0) rule(1,argnban0) rule(8,argnban0)
    rule(9,argnban0) rule(11,argnban0) rule(20,argnban0) rule(r8,argban0)
    inarg(cens,argban0) inarg(304,argban0) inarg(insf,argnban0) inarg(302,
    argnban0) inarg(304,argnban0) inarg(auth,argban0) inarg(over,argban0)
     inarg(fexp,argnban0) inarg(self,argnban0) inarg(weak,argnban0)
    derivable(fexp,argnban0) derivable(self,argnban0) derivable(auth,
    argban0) derivable(over,argban0) derivable(weak,argnban0) derivable(
    ban,argban0) derivable(need,argban0) derivable(insf,argban0)
    derivable(cens,argban0) derivable(304,argban0) derivable(nban,
    argnban0) derivable(alts,argnban0) derivable(deal,argnban0) derivable
    (insf,argnban0) derivable(302,argnban0) derivable(304,argnban0)
```

Encodings	Presentation
$\{300, 301, ..., 305\}$	$\{l_0, l_1, ..., l_5\}$
$\{1, 2, ..., 20\}$	$\{lr_1, lr_2, ..., lr_{20}\}$

Table 5. Name mapping between ASP-completion-generated learnable atoms and rules names and their figure labels used in the paper. Note that learnable rule identifiers $\{lr_1, lr_2, ..., lr_{20}\}$ indicate rules learned completely from scratch, however, existent rules previously not present in an argument may appear (be learned) in the completion of new arguments.

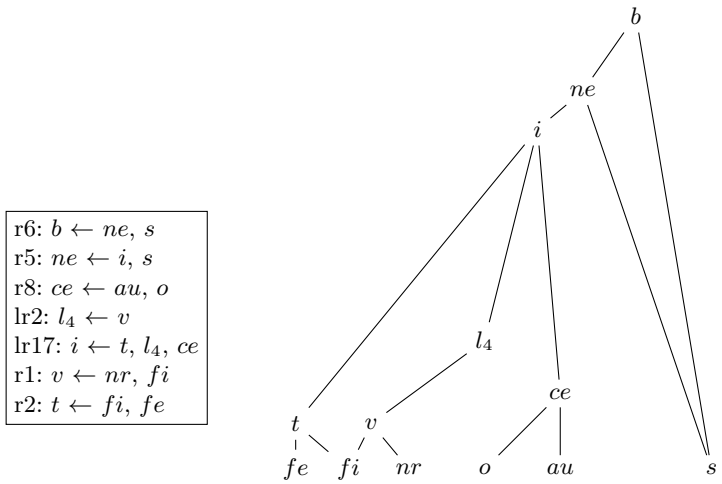


Fig. 4. Visual representation of the completion of the argument in Figure 2, with randomly assigned relatedness. Assumptions are on the bottom level, and derivation rules have as head some atom, and as body atoms appearing on lower levels and are connected to the head. On the left are the derivation rules, original and learned, with their identifier from the ASP completion based on the name mapping in Table 5. In this example, the identifier r8 is a rule originally available (in F) in the argument for nb, which was not learned from scratch, but used to complete the argument for b.

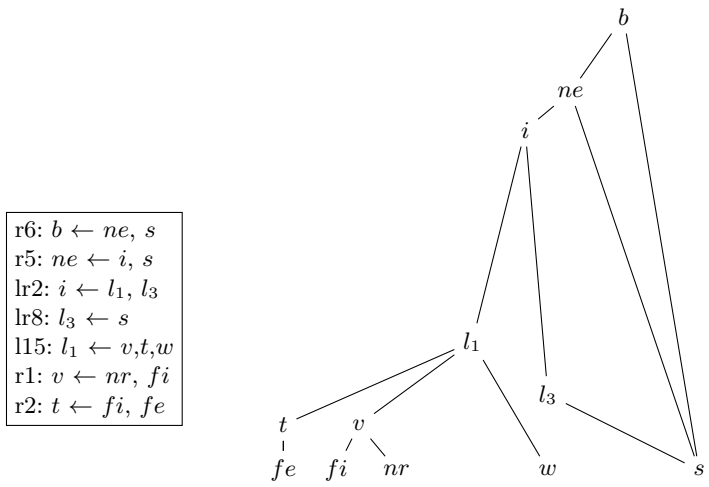


Fig. 5. Visual representation of the completion of the argument in Figure 2, with rule-based assigned relatedness.